VNRVJIET NAME OF THE LAB:_____.	Name of the Experiment:_____ Experiment No:____Date:_____

Week 5:
Aim: Implement the Client Server communication using Internet Domain Sockets and UNIX domain Sockets.

Aim: To implement client–server communication using Unix Domain Sockets (AF_UNIX) in C.

Description: Unix Domain Sockets enable communication between processes on the same system. The server creates, binds, and listens to a socket, while the client connects to it. The client sends data using write() and the server receives it using read().

Program:
Unix domain Server:

```
#include<stdlib.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/un.h>
#include<stdio.h>
#include<unistd.h>
#define NAME "socket243214"
int main()
{
   int sock,msgsock,rval;
   struct sockaddr_un server;
   char buf[1024];
   sock=socket(AF_UNIX,SOCK_STREAM,0);
   if(sock<0){
     perror("opening stream socket");
     exit(1);
    }
   server.sun_family=AF_UNIX;
   strcpy(server.sun_path,NAME);
   if(bind(sock,(struct sockaddr *)&server,sizeof(struct sockaddr_u$
{
  perror("binding stream socket");
exit(1);
}
 printf("Socket has name %s\n",server.sun_path);
listen(sock,5);
for(;;){
msgsock=accept(sock,0,0);
if(msgsock==-1)
perror("accept");
else do{
bzero(buf,sizeof(buf));
if((rval=read(msgsock,buf,1024))<0)
perror("reading stream message");
else if(rval==0)
printf("Ending connection\n");
else
printf("-->%s\n",buf);
}while(rval>0);
close(msgsock); }
close(sock);
unlink(NAME); }
```

VNRVJIET

NAME OF THE LAB:_____.

Name of the Experiment:_____

Experiment No:____Date:_____

Unix Domain Client:

```
#include<stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <stdio.h>
#include<unistd.h>
#define DATA "Half a league, half a league . . ."
int main(int argc,char* argv[])
 {
 int sock;
 struct sockaddr_un server;
 if (argc < 2)
 {
 printf("usage:%s <pathname>", argv[0]);
 exit(1);
}
sock = socket(AF_UNIX, SOCK_STREAM, 0);
 if (sock < 0) {
perror("opening stream socket");
 exit(1);
}
server.sun_family = AF_UNIX;
 strcpy(server.sun_path, argv[1]);
if (connect(sock, (struct sockaddr *) &server, sizeof(struct sockaddr_un)) < 0)
 {
 close(sock);
 perror("connecting stream socket");
 exit(1);
}
if (write(sock, DATA, sizeof(DATA)) < 0)
 perror("writing on stream socket");
 close(sock);
 return 0;
}
```

Output:

```
[24071A3214@rhel8-10 week4]$ gcc f2.c
[24071A3214@rhel8-10 week4]$ ./a.out
Socket has name socket243214
-->Half a league, half a league . . .
Ending connection
[24071A3214@rhel8-10 week4]$
```

```
[24071A3214@rhel8-10 week4]$ gcc f3.c
[24071A3214@rhel8-10 week4]$ ./a.out socket243214
[24071A3214@rhel8-10 week4]$
```

One program creates and names a socket (socket243214) and sends a message through it. The second program connects to the same socket and receives the message. The received text **"Half a league, half a league ..."** is displayed, and after the data transfer is completed, the connection ends successfully.

VNRVJIET NAME OF THE LAB:_____.	Name of the Experiment:_____ Experiment No:____Date:_____

Aim:To implement client–server communication using Internet Domain Sockets (AF_INET) in C.

Description: Internet Domain Sockets allow communication between processes over a network using IP address and port number. The client connects to the server and sends data, and the server receives and processes the message.

Program:

File4.c

```
#include <stdio.h>
#include <netinet/in.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define MAX 80
#define PORT 243214
#define SA struct sockaddr
void func(int sockfd)
{
   char buff[MAX];
   int n;
   for (;;)
   {
      bzero(buff, MAX);
      read(sockfd, buff, sizeof(buff));
      printf("From client: %s\t To client : ", buff);
      bzero(buff, MAX);
      n = 0;
      while ((buff[n++] = getchar()) != '\n');
      write(sockfd, buff, sizeof(buff));
      if (strncmp("exit", buff, 4) == 0)
         break;    }}
int main()
{
   int sockfd, connfd, len;
   struct sockaddr_in servaddr, cli;
   sockfd = socket(AF_INET, SOCK_STREAM, 0);
   if (sockfd == -1)
      exit(0);
   bzero(&servaddr, sizeof(servaddr));
   servaddr.sin_family = AF_INET;
   servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
 servaddr.sin_port = htons(PORT);
   if ((bind(sockfd, (SA *)&servaddr, sizeof(servaddr))) != 0)
      exit(0);
   if ((listen(sockfd, 5)) != 0)
      exit(0);
len = sizeof(cli);
   connfd = accept(sockfd, (SA *)&cli, &len);
   if (connfd < 0)
      exit(0);
   func(connfd);
   close(sockfd);}
```

VNRVJIET NAME OF THE LAB:_____.	Name of the Experiment:_____ Experiment No:____Date:_____

File5.c:

```
#include<stdio.h>
#include<netinet/in.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netdb.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#define MAX 80
#define PORT 3214
#define SA struct sockaddr
void func(int sockfd)
{
char buff[MAX];
int n;
for(;;){
bzero(buff,MAX);
read(sockfd,buff,sizeof(buff));
printf("From client: %s\t To client:",buff);
bzero(buff,MAX);
n=0;
while((buff[n++]=getchar())!='\n');
write(sockfd,buff,sizeof(buff));
if(strncmp("exit",buff,4)==0)
{
printf("Server Exit..\n");
break;
}
}
}
int main(){
int sockfd,connfd,len;
struct sockaddr_in servaddr,cli;
sockfd=socket(AF_INET,SOCK_STREAM,0);
if(sockfd==-1)
{
printf("socket creation failed...\n");
exit(0);
}
else
printf("Socket successfully created..\n");
bzero(&servaddr,sizeof(servaddr));
servaddr.sin_family=AF_INET;
servaddr.sin_addr.s_addr=htonl(INADDR_ANY);
servaddr.sin_port=htons(PORT);
if((bind(sockfd,(SA *)&servaddr,sizeof(servaddr)))!=0){
printf("socket bind failed...\n");
exit(0);
}
```

```
else
printf("socket successfully binded..\n");
if((listen(sockfd,5))!=0)
{
printf("Listen failed...\n");
exit(0);
}
else
printf("Server listening...\n");
len=sizeof(cli);
connfd=accept(sockfd,(SA *)&cli,&len);
if(connfd<0){
printf("Server accept failed...\n");
exit(0);
}
else
printf("server accepet the client..\n");
func(connfd);
close(sockfd);
}
```

Output:

```
[24071A3214@rhel8-10 week4]$ cc f4.c
[24071A3214@rhel8-10 week4]$ ./a.out
Socket successfully created..
socket successfully binded..
Server listening...
server accepet the client..
From client: hello
         To client:hi
From client: how are you doing
         To client:i am fine wbu
From client: i am fine
         To client:exit
Server Exit..
[24071A3214@rhel8-10 week4]$
```

```
[24071A3214@rhel8-10 week4]$ ./a.out
Socket successfully created.
Connected to server
Enter the string:hello
From Server: hi

Enter the string:how are you doing
From Server: i am fine wbu

Enter the string:i am fine
From Server: exit

Client exit...
[24071A3214@rhel8-10 week4]$
[24071A3214@rhel8-10 week4]$
```

The server first creates a socket, binds it, and starts listening for client connections. The client then creates its socket and connects to the server. After the connection is established, the client sends messages, and the server responds with appropriate replies. After the client enters exit and then the server types exit then it then the connection is ended.

VNRVJIET NAME OF THE LAB:_____.	Name of the Experiment:_____ Experiment No:____Date:_____

Aim: To implement client–server communication using UNIX domain sockets in C and exchange messages between client and server.

Description: This program demonstrates client–server communication using UNIX domain sockets. The server creates a socket and waits for a client to connect. The client connects to the server and both exchange messages until the word "exit" is entered.

Program:

Unix Domain Server

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<string.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/un.h>
#define NAME "socket3214"
#define MAX 1024
int main() {
   int sockfd, connfd;
   struct sockaddr_un server;
 char buff[MAX];
   int n;
   sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
   if (sockfd < 0) {
      perror("socket creation failed");
      exit(1);
   }
   server.sun_family = AF_UNIX;
   strcpy(server.sun_path, NAME);
   unlink(NAME);
   if (bind(sockfd, (struct sockaddr*)&server, sizeof(server)) < 0) {
      perror("bind failed");
      exit(1);
   }
   listen(sockfd, 5);
   printf("Socket created: %s\n", NAME);
   printf("Waiting for client...\n");
   connfd = accept(sockfd, NULL, NULL);
   printf("Client connected!\n");
   for (;;) {
      bzero(buff, MAX);
      read(connfd, buff, sizeof(buff));
      printf("From client: %s\tTo client : ", buff);
      if (strncmp("exit", buff, 4) == 0)
         break;
      bzero(buff, MAX);
      n = 0;
```

```
while ((buff[n++] = getchar()) != '\n');
        write(connfd, buff, sizeof(buff));

      if (strncmp("exit", buff, 4) == 0)
            break;
    }
    close(connfd);
    close(sockfd);
    unlink(NAME);
    return 0;
}
```

Unix Domain Client

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<sys/un.h>
#define MAX 1024
int main(int argc, char *argv[]) {
    int sockfd;
    struct sockaddr_un server;
    char buff[MAX];
    int n;
    if (argc != 2) {
 printf("Usage: %s <socketname>\n", argv[0]);
        exit(1);
    }
    sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket creation failed");
        exit(1);
    }
    server.sun_family = AF_UNIX;
    strcpy(server.sun_path, argv[1]);   // socket name from command line
    if (connect(sockfd, (struct sockaddr*)&server, sizeof(server)) < 0) {
        perror("connect failed");
        close(sockfd);
        exit(1);
    }
    printf("Connected to server!\n");
    for (;;) {
        bzero(buff, MAX);
        printf("Enter message: ");
        n = 0;
        while ((buff[n++] = getchar()) != '\n');
        write(sockfd, buff, sizeof(buff));
```

```
if (strncmp("exit", buff, 4) == 0)
        break;
    bzero(buff, MAX);
    read(sockfd, buff, sizeof(buff));
    printf("From server: %s\n", buff);
    if (strncmp("exit", buff, 4) == 0)
        break;
  }
  close(sockfd);
  return 0;}
```

Output:

```
[24071A3214@rhel8-10 week]$ gcc file1.c -o file1
[24071A3214@rhel8-10 week]$ gcc file2.c -o file2
[24071A3214@rhel8-10 week]$ ./file1
Socket created: socket3214
Waiting for client...
Client connected!
From client: hello
        To client : hi
From client: how are you
        To client : i am fine
From client: exit
        To client : [24071A3214@rhel8-10 week]$
```

```
[24071A3214@rhel8-10 week]$ ./file2 socket3214
Connected to server!
Enter message: hello
From server: hi

Enter message: how are you
From server: i am fine

Enter message: exit
[24071A3214@rhel8-10 week]$
```

The server successfully creates the socket and accepts the client connection. The client sends messages to the server and receives responses. When **"exit"** is entered, the communication ends and the connection is closed.